



**CONECTA
Y APRENDE**

COMPLEMENTO DE LA DOCUMENTACIÓN DE **app.py**

Aplicación **Conecta y Aprende**

Referencia técnica completa
de librerías, extensiones,
configuraciones, conceptos y
patrones utilizados en el archivo
app.py del proyecto.



LIBRERÍAS PRINCIPALES

Flask, PyMongo, Werkzeug,
datetime, os y más.



BASE DE DATOS USO DE MONGODB

Conexión, operaciones, patrones
y manejo de colecciones.



SEGURIDAD

Manejo seguro de contraseñas,
nombres de archivos y
control de acceso por roles.



CONFIGURACIONES Y ATRIBUTOS

Variables, rutas, permisos,
mapeos y constantes globales.



CONCEPTOS CLAVE Y FLUJOS

Sesiones, registros, actividades,
evidencias y administración.



GLOSARIO COMPLETO

Términos y atributos usados
en la aplicación.

```
app.py x
1  from flask import Flask, render_template, request,
2      redirect, url_for, session, flash
3  from werkzeug.security import generate_password_hash,
4      check_password_hash
5  from datetime import datetime
6  import os
7
8  # Configuración inicial
9  app = Flask(__name__)
10 app.secret_key = os.getenv("SECRET_KEY")
11 app.config['UPLOAD_FOLDER'] = 'static/uploads'
12 app.config['SQLALCHEMY_DATABASE_URI'] =
13     os.getenv('DATABASE_URL')
14 db = SQLAlchemy(app)
15
16 # Constantes de rol
17 ROL_ADMIN = 'admin'
18 ROL_INSTRUCTOR = 'instructor'
19 ROL_ESTUDIANTE = 'estudiante'
20 # ... resto del código ...
```



Contenido

Documentación de app.py	2
1. Librerías principales importadas en app.py	2
1.1 Flask	2
1.2 render_template	2
1.3 request	2
1.4 redirect	2
1.5 url_for	2
1.6 abort	2
1.7 session	2
1.8 flash	3
2. Base de datos: pymongo	3
2.1 MongoClient	3
2.2 ReturnDocument	3
2.3 Patrones de uso de MongoDB en el proyecto	3
3. BSON: ObjectId	3
4. Manejo seguro de contraseñas: werkzeug.security	3
4.1 generate_password_hash	3
4.2 check_password_hash	4
5. Seguridad en nombres de archivos: werkzeug.utils	4
6. Fechas y tiempo: datetime	4
7. Sistema de archivos: os	4
8. Configuración: config.Config	4
9. Extensiones y patrones usados en app.py	4
10. Atributos de configuración importantes	4
11. Principales conceptos de la aplicación	5
12. Flujo de base de datos y administrador por defecto	5
13. Glossario de términos y atributos	5

Documentación de app.py

Este documento explica en detalle las librerías, extensiones, configuraciones y conceptos usados en el archivo app.py del proyecto Conecta y Aprende. Está diseñado para ser editable y útil como referencia técnica.

1. Librerías principales importadas en app.py

1.1 Flask

- Flask es el framework web principal. En app.py se usa como motor de la aplicación.
1. Flask crea la aplicación web, administra rutas HTTP, maneja peticiones y renderiza plantillas.
 2. En el código: `app = Flask(__name__)`.

1.2 render_template

- Función de Flask para renderizar archivos HTML ubicados en la carpeta templates/.
3. Convierte un archivo HTML en respuesta HTTP usando variables de contexto.
 4. Ejemplo: `render_template("home.html", user=user, role_label=...)`.

1.3 request

- Objeto global que representa la petición HTTP actual.
5. Permite leer datos de formularios (`request.form`), parámetros de URL (`request.args`), archivos subidos (`request.files`) y método HTTP (`request.method`).
 6. Ejemplo: `request.form.get("email", "")`.

1.4 redirect

- Redirige al usuario a otra ruta dentro de la aplicación.
7. Devuelve una respuesta HTTP 302 con una ubicación nueva.
 8. Ejemplo: `redirect(url_for("home"))`.

1.5 url_for

- Construye URLs internas usando el nombre de la función vista.
9. Evita rutas rígidas en el código y hace la aplicación más mantenible.
 10. Ejemplo: `url_for("login")` genera la ruta `/login`.

1.6 abort

- Detiene la ejecución y devuelve un error HTTP inmediato.
11. Se usa para bloquear acceso no autorizado o recursos inexistentes.
 12. Ejemplo: `abort(403)` o `abort(404)`.

1.7 session

- Estructura de datos de Flask que guarda información entre peticiones.

13. Funciona con cookies seguras firmadas y permite mantener al usuario autenticado.

14. Ejemplo: `session["user_id"] = user_doc["id"]`.

1.8 flash

- Mensajería temporal que permite mostrar notificaciones de éxito o error al usuario.

15. Combina con plantillas para presentar mensajes luego de una redirección.

16. Ejemplo: `flash("Respuesta guardada correctamente.")`.

2. Base de datos: pymongo

2.1 MongoClient

- Conexión al servidor MongoDB.

17. Se usa para crear el cliente y obtener la base de datos.

18. Ejemplo: `client = MongoClient(app.config["MONGO_URI"])` y `db = client[app.config["MONGO_DB"]]`.

2.2 ReturnDocument

- Constante de PyMongo para controlar el documento retornado por `find_one_and_update`.

19. Se usa en `get_next_sequence` para obtener el documento actualizado después del incremento.

20. Ejemplo: `return_document=ReturnDocument.AFTER`.

2.3 Patrones de uso de MongoDB en el proyecto

- `insert_one(...)`, `find_one(...)`, `find(...).sort(...)`, `update_one(...)`, `delete_one(...)`, `delete_many(...)`, `count_documents(...)`, `aggregate(...)`, `create_index(...)`.

21. Estas funciones manipulan colecciones, consultan datos y crean índices para mejorar el rendimiento.

3. BSON: ObjectId

- `ObjectId` es el tipo especial usado por MongoDB para el campo `_id`.

22. Se usa para convertir cadenas de texto en identificadores internos de MongoDB.

23. Ejemplo: `message_doc = db.contact_messages.find_one({"_id": ObjectId(message_id)})`.

4. Manejo seguro de contraseñas: werkzeug.security

4.1 generate_password_hash

- Convierte una contraseña de texto plano en un hash seguro.

24. Se almacena en la base de datos, nunca la contraseña original.

25. Ejemplo: `generate_password_hash(password)`.

4.2 check_password_hash

- Valida si una contrase?a coincide con el hash almacenado.

26. Evita comparar contrase?as en texto plano.

27. Ejemplo: `check_password_hash(user_doc["password"], password)`.

5. Seguridad en nombres de archivos: `werkzeug.utils`

- `secure_filename` normaliza nombres de archivos subidos para evitar caracteres peligrosos.

28. Protege contra rutas maliciosas y caracteres que podr?an romper el almacenamiento.

29. Ejemplo: `filename = secure_filename(image_file.filename)`.

6. Fechas y tiempo: `datetime`

- `datetime` se usa para marcas de tiempo exactas con fecha y hora.

30. `date` representa solo la fecha sin hora, ?til para c?lculos de plazos y validaciones.

31. `timedelta` representa un intervalo de tiempo entre fechas.

7. Sistema de archivos: `os`

- `os.path.join` construye rutas de forma segura entre carpetas.

32. `os.makedirs` crea carpetas si no existen.

8. Configuraci?n: `config.Config`

- `Config` es una clase propia definida en `config.py` que contiene variables de configuraci?n globales.

33. Campos importantes: `SECRET_KEY`, `MONGO_URI`, `MONGO_DB`.

34. `app.config.from_object(Config)` carga estas variables en la aplicaci?n Flask.

9. Extensiones y patrones usados en `app.py`

- No se utilizan extensiones externas de Flask como Flask-Login, Flask-WTF o Flask-Mail.

35. El proyecto usa caracter?sticas nativas de Flask junto con PyMongo para la base de datos.

36. Las rutas, el manejo de errores y el contexto global se implementan directamente con Flask.

10. Atributos de configuraci?n importantes

- `UPLOAD_FOLDER`: Carpeta f?sica para almacenar archivos subidos.
- `ALLOWED_EXTENSIONS`: Extensiones permitidas para im?genes.
- `ROLE_LABELS`, `ROLE_PAGES`, `PAGE_TITLES`, `PAGE_TEMPLATES`, `PAGE_SUMMARIES`, `ROLE_ACTIONS`: Mapa y permisos de navegaci?n por rol.

11. Principales conceptos de la aplicaci3n

- Control de acceso por rol: Se verifica user["role"] antes de permitir rutas administrativas.
- Gesti3n de sesi3n: login almacena user_id y role, logout borra la sesi3n.
- Registro de usuarios: guarda cuentas con active=False y contrase#a hasheada.
- Gesti3n de actividad y evidencias: el rol recuperacion puede solicitar actividades y subir evidencias semanales.
- Administraci3n de recursos: rutas /admin/... para ver usuarios, solicitudes, transferencias, pr3stamos y certificados.

12. Flujo de base de datos y administrador por defecto

- La funci3n init_db() crea 3ndices y asegura que exista un administrador predeterminado.

37. ensure_default_admin() crea la cuenta admin@conectayaprende.local con contrase#a admin123 si no existe.

13. Glossario de terminos y atributos

Termino	Significado detallado
app	Instancia principal de Flask que contiene la aplicaci3n web.
route	Ruta URL asociada a una funci3n vista.
endpoint	Nombre interno de una vista Flask, usado por url_for.
render_template	Funci3n que procesa un HTML con variables y devuelve respuesta.
request	Representa la petici3n HTTP entrante.
session	Almacena datos de usuario entre peticiones usando cookies firmadas.
flash	Mensajes temporales que el servidor env?a al cliente para mostrar alertas.
redirect	Respuesta que env?a al navegador a otra URL.
url_for	Genera la URL asociada a un endpoint Flask.
abort	Corta el flujo y devuelve un error HTTP espec?fico.
MongoClient	Cliente de PyMongo para conectar con MongoDB.
db	Referencia a la base de datos Mongo activa en la aplicaci3n.
collection	Tabla de MongoDB; por ejemplo db.users, db.notifications.
document	Registro de MongoDB, equivalente a una fila JSON.
find_one	Busca un documento que coincida con un criterio.
insert_one	Inserta un nuevo documento en una colecci3n.
update_one	Actualiza un documento existente.
delete_one / delete_many	Elimina documento(s) de la colecci3n.
count_documents	Cuenta los documentos que cumplen una condici3n.
aggregate	Ejecuta un pipeline de agregaci3n para sumar o agrupar datos.
create_index	Crea 3ndices para acelerar consultas.
upsert	Inserta si no existe, o actualiza si ya existe.

ReturnDocument.AFTER	Devuelve el documento despu?s de aplicar la actualizaci?n.
ObjectId	Tipo especial de Mongo usado para el campo _id.
generate_password_hash	Crea un hash seguro de contrase?a.
check_password_hash	Valida una contrase?a contra un hash almacenado.
secure_filename	Sanitiza un nombre de archivo para evitar problemas de seguridad.
datetime.utcnow()	Fecha y hora actual en UTC.
date.fromisoformat()	Convierte una cadena YYYY-MM-DD en un objeto date.
isoformat()	Convierte una fecha a cadena YYYY-MM-DD.
timedelta	Intervalo de tiempo entre fechas.
os.path.dirname(_file_)	Directorio donde est? el archivo app.py.
os.makedirs(..., exist_ok=True)	Crea carpetas si no est?n presentes.
UPLOAD_FOLDER	Carpeta f?sica donde se guardan archivos subidos.
ALLOWED_EXTENSIONS	Tipos de archivo permitidos para carga.
ROLE_PAGES	Mapa que define qu? p?ginas puede ver cada rol.
ROLE_LABELS	Traducci?n de roles internos a nombres legibles.
PAGE_TEMPLATES	Plantillas HTML que muestran cada secci?n de interfaz.
PAGE_SUMMARIES	Res?menes descritos para cada p?gina.
user_id	Identificador interno del usuario en la base de datos.
request_data	Documento de solicitud de actividad del usuario.
evidence_entries	Registro de evidencias semanales subidas.
penalty_dates	Semanas en que no se subi? evidencia y se aplic? penalizaci?n.
progress	Porcentaje de avance calculado para la actividad.
active_date	Fecha de referencia usada para el calendario del programa.